Ain Shams University

Faculty of Engineering
Computer and Systems Engineering Department
CHEP – Spring 2026

Secure Communication Suite

A Cryptography Application in Python

CSE451: Computer and Network Security

Group Members:

Marwan Noureldin – 21P0165
Yehia Mahmoud – 21P0166
Ahmed Ayman – 21P002
Youssef Ayman – 21P0010
Hanna Mohamed – 21P0162

April 2026

Contents

1 Phase 2: Development of Cryptographic Modules

This phase focuses on implementing the three fundamental cryptographic building blocks of the Secure Communication Suite: the **Block Cipher Module** (AES), the **Public Key Cryptosystem Module** (RSA), and the **Hashing Module** (SHA-256). Each module is encapsulated in its own Python class inside the `src/crypto/` package.

1.1 Block Cipher Module – AES (`block_cipher.py`)

1.1.1 Overview

The Advanced Encryption Standard (AES) is a symmetric-key block cipher adopted as the encryption standard by the U.S. National Institute of Standards and Technology (NIST). It operates on fixed-size blocks of 128 bits and supports key sizes of 128, 192, or 256 bits.

Our implementation uses **AES-128** (16-byte key) with the **EAX mode of operation**. EAX is an authenticated encryption mode, meaning that it simultaneously provides both *confidentiality* and *integrity/authenticity*. This is achieved by producing a *nonce* and an authentication *tag* alongside the ciphertext.

1.1.2 Why EAX Mode?

- **Authenticated Encryption:** Unlike ECB or CBC modes, EAX guarantees that any tampering with the ciphertext will be detected during decryption, because the tag verification will fail.
- **No IV Management Burden:** EAX automatically generates and manages a unique nonce for each encryption call, preventing nonce-reuse vulnerabilities.
- **Stream-Friendly:** EAX can encrypt data of arbitrary length; no padding is needed, eliminating padding-oracle attack vectors.

1.1.3 Implementation Details

```python
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

class AESCipher:
    """
    AES encryption module using EAX mode.
    """
    def __init__(self, key: bytes = None):
        if key is None:
            self.key = get_random_bytes(16)  # AES-128
        else:
            if len(key) not in [16, 24, 32]:
                raise ValueError("Key must be 16, 24, or 32 bytes")
            self.key = key

    def encrypt(self, plaintext: bytes) -> bytes:
        """
        Encrypts plaintext using AES-EAX mode.
        Returns nonce + tag + ciphertext.
        """
```

```python
        cipher = AES.new(self.key, AES.MODE_EAX)
        nonce = cipher.nonce
        ciphertext, tag = cipher.encrypt_and_digest(plaintext)
        return nonce + tag + ciphertext

    def decrypt(self, encrypted_data: bytes) -> bytes:
        """
        Decrypts data encrypted by the encrypt method.
        """
        nonce = encrypted_data[:16]
        tag = encrypted_data[16:32]
        ciphertext = encrypted_data[32:]
        cipher = AES.new(self.key, AES.MODE_EAX, nonce=nonce)
        plaintext = cipher.decrypt_and_verify(ciphertext, tag)
        return plaintext
```

Listing 1: AES Block Cipher Module – `block_cipher.py`

1.1.4 How the Block Cipher Module Works

1. **Key Initialization:** The constructor accepts an optional key parameter. If no key is provided, a cryptographically secure random 16-byte key is generated using `get_random_bytes()`. If a key is supplied, its length is validated to be 16, 24, or 32 bytes (corresponding to AES-128, AES-192, or AES-256 respectively).

2. **Encryption Process:**
   (a) A new AES cipher object is created in EAX mode, which internally generates a fresh 16-byte **nonce**.
   (b) The plaintext is encrypted and a 16-byte **authentication tag** is computed via `encrypt_and_digest()`.
   (c) The output is a single byte string structured as: `nonce (16 B)` ‖ `tag (16 B)` ‖ `ciphertext`.

3. **Decryption Process:**
   (a) The received byte string is split into nonce (first 16 bytes), tag (next 16 bytes), and ciphertext (remainder).
   (b) A new AES cipher is initialized with the *same key* and the extracted nonce.
   (c) `decrypt_and_verify()` recovers the plaintext and simultaneously verifies the tag. If the data has been tampered with, a `ValueError` is raised.

1.1.5 Data Format

The encrypted output follows a fixed layout, as summarized in Table **??**.

Table 1: AES-EAX encrypted data format

Field	Size (bytes)	Purpose
Nonce	16	Ensure unique ciphertexts
Tag	16	Authenticate ciphertext
Ciphertext	Variable	Encrypted payload

1.2 Public Key Cryptosystem Module – RSA (`public_key.py`)

1.2.1 Overview

RSA (Rivest–Shamir–Adleman) is the most widely deployed public-key cryptosystem. Its security relies on the computational difficulty of factoring the product of two large prime numbers. In our suite, RSA is used specifically for **secure key exchange**: the client encrypts a randomly generated AES session key with the server's RSA public key, and only the server (which possesses the corresponding private key) can decrypt it.

1.2.2 PKCS#1 OAEP Padding

Our implementation uses **PKCS#1 OAEP** (Optimal Asymmetric Encryption Padding), which is the recommended RSA encryption scheme. OAEP provides:

- **Semantic Security:** Identical plaintext encrypted multiple times produces different ciphertext, thwarting chosen-plaintext attacks.
- **Integrity:** The padding scheme includes a hash-based verification that detects malformed ciphertexts.

1.2.3 Implementation Details

```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

class RSACipher:
    """
    RSA asymmetric encryption module for secure key sharing.
    """
    @staticmethod
    def generate_keys(key_size=2048):
        """Generates an RSA keypair (private, public) as bytes."""
        key = RSA.generate(key_size)
        private_key = key.export_key()
        public_key = key.publickey().export_key()
        return private_key, public_key

    @staticmethod
    def encrypt(public_key: bytes, message: bytes) -> bytes:
        """Encrypts a message using a public RSA key."""
        rsa_key = RSA.import_key(public_key)
        cipher = PKCS1_OAEP.new(rsa_key)
        return cipher.encrypt(message)

    @staticmethod
    def decrypt(private_key: bytes, ciphertext: bytes) -> bytes:
        """Decrypts ciphertext using a private RSA key."""
        rsa_key = RSA.import_key(private_key)
        cipher = PKCS1_OAEP.new(rsa_key)
        return cipher.decrypt(ciphertext)
```

Listing 2: RSA Public Key Module – `public_key.py`

1.2.4 Role of the Public Key Cryptosystem Module

The RSA module fulfils two critical roles in the Secure Communication Suite:

1. **Key Generation:** The `generate_keys()` method produces a 2048-bit RSA key pair. The private key is stored securely on the server, while the public key is transmitted to every connecting client. The 2048-bit key size provides a security level of approximately 112 bits, which is considered adequate for current use.

2. **Hybrid Encryption Bridge:** Pure RSA encryption is limited to small messages (at most 245 bytes with 2048-bit keys and OAEP). Therefore, RSA is used only to securely transmit a 16-byte AES session key from the client to the server, after which all bulk data encryption is handled by the much faster AES algorithm. This is the classical *hybrid encryption* pattern.

3. **Static Methods:** All methods are `@staticmethod`, which means the class is used as a stateless utility. Keys are passed explicitly, improving testability and allowing integration with the `KeyManager`.

1.3 Hashing Module – SHA-256 (`hashing.py`)

1.3.1 Overview

SHA-256 (Secure Hash Algorithm, 256-bit) is a member of the SHA-2 family designed by the NSA. It produces a fixed-size 256-bit (32-byte) digest from an arbitrary-length input. The algorithm guarantees:

- **Pre-image Resistance:** Given a hash h, it is computationally infeasible to find any message m such that SHA-256$(m) = h$.

- **Second Pre-image Resistance:** Given a message m_1, it is infeasible to find a different m_2 where SHA-256(m_1) = SHA-256(m_2).

- **Collision Resistance:** It is infeasible to find any two messages $m_1 \neq m_2$ with the same hash.

1.3.2 Implementation Details

```python
from Crypto.Hash import SHA256

class DeepHash:
    """
    Data integrity checking using SHA-256.
    """
    @staticmethod
    def hash_data(data: bytes) -> str:
        """Computes the SHA-256 hash of the given data."""
        h = SHA256.new()
        h.update(data)
        return h.hexdigest()

    @staticmethod
    def verify(data: bytes, expected_hash: str) -> bool:
        """Verifies SHA-256 hash of data matches expected hash."""
```

```
        return DeepHash.hash_data(data) == expected_hash
```

Listing 3: SHA-256 Hashing Module – `hashing.py`

1.3.3 How the Hashing Module Ensures Data Integrity

1. **Digest Computation – `hash_data()`:** The method instantiates a SHA-256 hasher, feeds in the binary data via `update()`, and returns the hexadecimal-encoded digest string (64 hex characters).

2. **Verification – `verify()`:** To check whether data has been altered, the caller provides the data and the previously known hash. The method recomputes the hash and performs a simple equality comparison. If the hashes differ, the data has been tampered with.

3. **Usage in the Suite:**

   - **Password Storage:** The `AuthManager` (Phase 3) uses SHA-256 with a random salt to hash passwords before storing them. This ensures that even if the user database is compromised, plaintext passwords are not revealed.
   - **GUI Integrity Tool:** The graphical interface provides a "Data Integrity" view where users can manually compute SHA-256 digests of arbitrary text, enabling them to verify file or message integrity out-of-band.
   - **AES-EAX Tag:** The AES-EAX authentication tag (Phase 2) internally relies on CMAC, but the SHA-256 module serves as an independent integrity layer available to the user.

1.3.4 Phase 2 Summary

By the end of Phase 2, the three core cryptographic modules are complete:

- **AES-EAX** provides fast, authenticated symmetric encryption.
- **RSA-OAEP** provides secure asymmetric key exchange.
- **SHA-256** provides tamper-evident hashing.

2 Phase 3: Development of Key Management and Authentication Modules

Phase 3 builds the trust infrastructure that underpins the entire suite. The **Key Management Module** handles generation, storage, and distribution of RSA key pairs, while the **Authentication Module** implements password-based user authentication with salted hashing. Both modules reside in the `src/auth/` package.

2.1 Key Management Module (`key_management.py`)

2.1.1 Overview

Cryptographic keys are the most sensitive asset in any security system. If a private key is compromised, all data encrypted with the corresponding public key can be decrypted. The `KeyManager` class addresses three concerns:

1. **Generation:** Creating RSA key pairs when they do not yet exist.
2. **Storage:** Persisting keys to the filesystem in PEM format.
3. **Distribution:** Sharing public keys between communicating parties.

2.1.2 Implementation Details

```python
import os
from crypto.public_key import RSACipher

class KeyManager:
    """Secure methods for key generation, distribution, and storage."
    """

    @staticmethod
    def get_or_create_keys(user_id: str, key_dir="keys"):
        if not os.path.exists(key_dir):
            os.makedirs(key_dir)

        priv_path = os.path.join(key_dir, f"{user_id}_private.pem")
        pub_path  = os.path.join(key_dir, f"{user_id}_public.pem")

        if os.path.exists(priv_path) and os.path.exists(pub_path):
            with open(priv_path, "rb") as f:
                priv = f.read()
            with open(pub_path, "rb") as f:
                pub = f.read()
            return priv, pub

        priv, pub = RSACipher.generate_keys()

        with open(priv_path, "wb") as f:
            f.write(priv)
        with open(pub_path, "wb") as f:
            f.write(pub)

        return priv, pub
```

```python

    @staticmethod
    def save_partner_key(partner_id: str, pub_key: bytes,
                         key_dir="keys"):
        if not os.path.exists(key_dir):
            os.makedirs(key_dir)
        path = os.path.join(key_dir, f"{partner_id}_public.pem")
        with open(path, "wb") as f:
            f.write(pub_key)

    @staticmethod
    def load_partner_key(partner_id: str,
                         key_dir="keys") -> bytes:
        path = os.path.join(key_dir, f"{partner_id}_public.pem")
        if not os.path.exists(path):
            return None
        with open(path, "rb") as f:
            return f.read()
```

Listing 4: Key Management Module – `key_management.py`

2.1.3 How the Key Management Module Secures Key Distribution and Storage

1. **Lazy Key Generation (`get_or_create_keys`):**
   - On first invocation for a given `user_id`, the method generates a fresh 2048-bit RSA key pair via `RSACipher.generate_keys()` and persists both the private and public key to disk in PEM format.
   - On subsequent invocations, the method reads the existing keys from disk, avoiding redundant key generation and ensuring key continuity across server restarts.
2. **Key Storage:**
   - All keys are stored in a dedicated `keys/` directory, created automatically if absent.
   - Naming convention: `<user_id>_private.pem` and `<user_id>_public.pem`, making key identification straightforward.
   - Keys are stored in the standard **PEM** (Privacy Enhanced Mail) format, which is Base64-encoded DER wrapped with header and footer lines. This format is human-readable and widely interoperable.
3. **Partner Key Distribution (`save_partner_key` / `load_partner_key`):**
   - When a client connects, the server sends its public key. The client calls `save_partner_key("server", pub_key)` to persist this public key locally.
   - `load_partner_key()` retrieves a previously saved partner's public key. This enables subsequent sessions to verify the server's identity.
4. **Security Considerations:**

- Private keys remain exclusively on the machine that generated them; they are never transmitted over the network.
- In a production environment, the `keys/` directory should be protected with strict file-system permissions (e.g., `chmod 700`).

2.2 Authentication Module (`authentication.py`)

2.2.1 Overview

The authentication module implements a **password-based authentication** system that securely stores credentials and verifies user identities. It uses **salted SHA-256 hashing** to protect stored passwords.

2.2.2 Implementation Details

```python
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes
import json
import os

class AuthManager:
    """Implement password-based authentication mechanisms."""
    def __init__(self, db_path="users.json"):
        self.db_path = db_path
        self._load_users()

    def _load_users(self):
        if not os.path.exists(self.db_path):
            self.users = {}
            self._save_users()
        else:
            with open(self.db_path, "r") as f:
                self.users = json.load(f)

    def _save_users(self):
        with open(self.db_path, "w") as f:
            json.dump(self.users, f)

    def _hash_password(self, password: str, salt: str) -> str:
        h = SHA256.new()
        h.update((password + salt).encode('utf-8'))
        return h.hexdigest()

    def register(self, username: str, password: str) -> bool:
        if username in self.users:
            return False
        salt = get_random_bytes(16).hex()
        hashed = self._hash_password(password, salt)
        self.users[username] = {"salt": salt, "hash": hashed}
        self._save_users()
        return True

    def authenticate(self, username: str, password: str) -> bool:
        if username not in self.users:
            return False
```

```
        user_data = self.users[username]
        hashed = self._hash_password(password, user_data["salt"])
        return hashed == user_data["hash"]
```

Listing 5: Authentication Module – `authentication.py`

2.2.3 Authentication Mechanisms

The module provides two operations:

Registration.

1. The user supplies a `username` and `password`.
2. The system checks whether the username already exists; if so, registration is rejected to prevent duplicates.
3. A cryptographically secure random **salt** (16 bytes, hex-encoded to 32 characters) is generated using `get_random_bytes(16)`.
4. The password is concatenated with the salt, and the result is hashed with SHA-256: $h = \text{SHA-256}(\texttt{password} \| \texttt{salt})$.
5. The user record `{"salt": ..., "hash": ...}` is stored in a JSON file (`users.json`).

Login.

1. The user provides `username` and `password`.
2. The system retrieves the stored salt for that username.
3. It recomputes $h' = \text{SHA-256}(\texttt{password} \| \texttt{salt})$ and compares h' with the stored hash.
4. If they match, the user is authenticated; otherwise, login fails.

2.2.4 How the Authentication Module Verifies User Identities

- **Salted Hashing Prevents Rainbow-Table Attacks:** Because each user has a unique random salt, precomputed hash tables (rainbow tables) cannot be used to reverse the hash.
- **No Plaintext Storage:** Passwords are *never* stored in plaintext. Even if an attacker gains access to `users.json`, they obtain only salts and hashes, not usable passwords.
- **Duplicate Session Prevention:** The server (`server.py`) maintains a dictionary `self.clients` mapping connected sockets to usernames. Before authenticating a login, the server checks whether the username already appears in the dictionary. If so, it sends a `"DUP"` response and refuses the connection, preventing simultaneous duplicate sessions.
- **Persistence:** The JSON-based user database is loaded at server startup and updated transactionally on each registration, ensuring data survives server restarts.

2.2.5 Phase 3 Summary

By the end of Phase 3:

- The **KeyManager** handles RSA key lifecycle (generate, store, load, distribute).
- The **AuthManager** provides secure registration and login with salted SHA-256 password hashing.
- Together, these modules provide the trust foundation for Phase 4's integrated system.

3 Phase 4: Integration and Testing

Phase 4 brings all modules together into a fully functional **Secure Communication Suite**. This phase covers the `server.py` (multi-client secure server), `client.py` (client-side networking), and `gui.py` (graphical user interface), as well as the testing methodology.

3.1 System Architecture

The suite follows a classic **client–server** architecture augmented with a hybrid cryptographic protocol:

1. **Server** (`server.py`): Listens for incoming TCP connections, authenticates users, performs RSA key exchange, and relays AES-encrypted messages between clients.
2. **Client** (`client.py`): Connects to the server, performs authentication and key exchange, then sends/receives AES-encrypted messages.
3. **GUI** (`gui.py`): Provides a modern graphical front-end built with CustomTkinter, featuring login/register screens, secure chat, and cryptographic tools.

Communication Protocol Flow

$$\texttt{Client} \xrightarrow{\text{1. LOGIN/REGISTER + credentials}} \texttt{Server}$$

$$\texttt{Client} \xleftarrow{\text{2. OK / ERR / DUP}} \texttt{Server}$$

$$\texttt{Client} \xleftarrow{\text{3. RSA Public Key}} \texttt{Server}$$

$$\texttt{Client} \xrightarrow{\text{4. RSA-encrypted AES key}} \texttt{Server}$$

$$\texttt{Client} \xleftarrow{\text{5. READY}} \texttt{Server}$$

`Client` 6. AES-EAX encrypted messages `Server`

Figure 1: Protocol flow for secure session establishment

3.2 Server Module (`server.py`)

3.2.1 Implementation Details

```python
import socket
import threading
from auth.authentication import AuthManager
from auth.key_management import KeyManager
from crypto.public_key import RSACipher
from crypto.block_cipher import AESCipher

class SecureServer:
    def __init__(self, host='127.0.0.1', port=65432):
        self.host = host
        self.port = port
        self.auth = AuthManager()
        self.server_socket = socket.socket(
            socket.AF_INET, socket.SOCK_STREAM)
```

```python
        self.server_socket.setsockopt(
            socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.server_socket.bind((self.host, self.port))
        self.priv_key, self.pub_key = \
            KeyManager.get_or_create_keys("server")
        self.clients = {}            # {socket: username}
        self.broadcast_ciphers = {}  # {socket: AESCipher}

    def start(self):
        self.server_socket.listen()
        print(f"[*] Server listening on {self.host}:{self.port}")
        try:
            while True:
                conn, addr = self.server_socket.accept()
                thread = threading.Thread(
                    target=self.handle_client, args=(conn, addr))
                thread.start()
        except KeyboardInterrupt:
            print("[*] Server shutting down")
            self.server_socket.close()

    def broadcast(self, message_bytes: bytes, sender_conn):
        for client_conn, cipher in \
                self.broadcast_ciphers.items():
            if client_conn != sender_conn:
                try:
                    encrypted = cipher.encrypt(message_bytes)
                    client_conn.send(encrypted)
                except Exception as e:
                    print(f"[-] Broadcast error: {e}")

    def handle_client(self, conn, addr):
        print(f"[*] Connection accepted from {addr}")
        try:
            # 1. Authentication
            auth_msg = conn.recv(1024).decode()
            action, username, password = \
                auth_msg.split('||')

            if action == 'REGISTER':
                success = self.auth.register(
                    username, password)
                conn.send(b"OK" if success else b"ERR")
                if not success: return
            elif action == 'LOGIN':
                if username in self.clients.values():
                    conn.send(b"DUP")
                    return
                success = self.auth.authenticate(
                    username, password)
                conn.send(b"OK" if success else b"ERR")
                if not success: return
            else:
                conn.send(b"ERR")
                return

            # 2. Key Exchange
            conn.send(self.pub_key)
```

```python
            encrypted_aes_key = conn.recv(1024)
            aes_key = RSACipher.decrypt(
                self.priv_key, encrypted_aes_key)
            cipher = AESCipher(aes_key)

            self.clients[conn] = username
            self.broadcast_ciphers[conn] = cipher
            conn.send(b"READY")

            self.broadcast(
                f"[SERVER] {username} joined!".encode(),
                conn)

            # 3. Secure Communication Loop
            while True:
                encrypted_data = conn.recv(4096)
                if not encrypted_data:
                    break
                plaintext = cipher.decrypt(encrypted_data)
                formatted = \
                    f"[{username}]: {plaintext.decode()}"
                self.broadcast(formatted.encode(), conn)

        except Exception as e:
            print(f"[-] Error handling {addr}: {e}")
        finally:
            username = self.clients.get(conn, "Unknown")
            self.clients.pop(conn, None)
            self.broadcast_ciphers.pop(conn, None)
            conn.close()
            self.broadcast(
                f"[SERVER] {username} left.".encode(), None)

if __name__ == '__main__':
    server = SecureServer()
    server.start()
```

Listing 6: Secure Server – `server.py`

3.2.2 How Modules Are Integrated in the Server

1. **AuthManager Integration:** At startup, an `AuthManager` instance is created, loading the user database. Each incoming connection is first processed through `register()` or `authenticate()`.

2. **KeyManager Integration:** On initialization, the server calls `KeyManager.get_or_create_keys` to obtain its RSA key pair. The public key is sent to each client during the handshake.

3. **RSACipher Integration:** After the client receives the server's public key, it encrypts a freshly generated AES session key and sends it back. The server uses `RSACipher.decrypt()` to recover the AES key.

4. **AESCipher Integration:** A per-client `AESCipher` object is created with the negotiated session key. All subsequent messages are encrypted and decrypted through this cipher.

5. **Multi-threaded Design:** Each client connection is handled in a separate thread, enabling concurrent secure sessions. The `broadcast()` method iterates over all connected clients (except the sender) and encrypts the message with each client's individual AES cipher.

3.3 Client Module (`client.py`)

```python
import socket
import threading
from auth.key_management import KeyManager
from crypto.public_key import RSACipher
from crypto.block_cipher import AESCipher
from Crypto.Random import get_random_bytes

class SecureClient:
    def __init__(self, host='127.0.0.1', port=65432):
        self.host = host
        self.port = port
        self.socket = None
        self.aes_cipher = None
        self.connected = False
        self.username = None

    def connect_and_auth(self, action, username, password):
        try:
            self.socket = socket.socket(
                socket.AF_INET, socket.SOCK_STREAM)
            self.socket.connect((self.host, self.port))

            # 1. Auth Phase
            msg = f"{action}||{username}||{password}"
            self.socket.send(msg.encode())
            res = self.socket.recv(1024).decode()
            if res == "DUP":
                self.socket.close()
                return False, "User already logged in."
            if res != "OK":
                self.socket.close()
                return False, "Authentication failed."

            # 2. Key Exchange Phase
            server_pub_key = self.socket.recv(2048)
            KeyManager.save_partner_key(
                "server", server_pub_key)

            aes_key = get_random_bytes(16)
            encrypted_aes_key = RSACipher.encrypt(
                server_pub_key, aes_key)
            self.socket.send(encrypted_aes_key)

            ready = self.socket.recv(1024)
            if ready == b"READY":
                self.aes_cipher = AESCipher(aes_key)
                self.connected = True
                self.username = username
                return True, "Secure session established."

```

```python
            return False, "Session setup failed."
        except Exception as e:
            if self.socket:
                self.socket.close()
            return False, f"Connection error: {e}"

    def send_message(self, message: str) -> bool:
        if not self.connected:
            return False
        try:
            encrypted = self.aes_cipher.encrypt(
                message.encode())
            self.socket.send(encrypted)
            return True
        except:
            self.connected = False
            return False

    def receive_message(self) -> str:
        if not self.connected:
            return None
        try:
            encrypted = self.socket.recv(4096)
            if not encrypted:
                self.connected = False
                return None
            plaintext = self.aes_cipher.decrypt(encrypted)
            return plaintext.decode()
        except:
            self.connected = False
            return None

    def close(self):
        if self.socket:
            self.socket.close()
        self.connected = False
```

Listing 7: Secure Client – `client.py`

3.4 Graphical User Interface (`gui.py`)

The GUI is built using **CustomTkinter**, a modern Python UI library that extends tkinter with contemporary styling. The interface consists of two main screens:

1. **Authentication Screen:** A centred login card with username/password fields and Login/Register buttons.
2. **Main Interface:** A sidebar-based layout with:
   - **Secure Chat** – real-time encrypted messaging with a text display, message entry, and send button.
   - **Data Integrity (Crypto Tools)** – an SHA-256 hasher and a standalone AES encrypt/decrypt tool.
   - **Disconnect** – graceful session teardown.

```python
import threading
import customtkinter as ctk
import tkinter.messagebox as messagebox
from client import SecureClient
from crypto.hashing import DeepHash

ctk.set_appearance_mode("System")
ctk.set_default_color_theme("blue")

class SecureApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Secure Communication Suite")
        self.geometry("900x600")
        self.client = SecureClient()
        self.receive_thread = None
        self.running = True
        self.show_auth_frame()

    def authenticate(self, action):
        user = self.username_entry.get()
        pwd = self.password_entry.get()
        if not user or not pwd:
            messagebox.showwarning("Input Error",
                "Provide both username and password")
            return
        success, msg = self.client.connect_and_auth(
            action, user, pwd)
        if success:
            self.show_main_interface()
            self.start_receiving()
        else:
            messagebox.showerror("Auth Error", msg)

    def send_message(self):
        msg = self.msg_entry.get()
        if msg:
            success = self.client.send_message(msg)
            if success:
                self.msg_entry.delete(0, 'end')
                self.append_chat(f"You: {msg}")
            else:
                messagebox.showerror("Error",
                    "Failed to send. Lost connection.")
                self.disconnect()

    def receive_loop(self):
        while self.running and self.client.connected:
            msg = self.client.receive_message()
            if msg:
                self.after(0, self.append_chat, msg)
            else:
                if self.running:
                    self.after(0, self.disconnect)
                break

if __name__ == "__main__":
```

```
    app = SecureApp()
    app.mainloop()
```

Listing 8: GUI Application – `gui.py` (excerpt)

3.5 How the Suite Secures Internet Services

The Secure Communication Suite applies its cryptographic modules to provide defence-in-depth for network communication:

1. **Data in Transit:** All messages exchanged between client and server are encrypted using AES-EAX with per-session keys. This prevents eavesdropping (confidentiality) and detects any modification of packets (integrity).
2. **Key Exchange Security:** The AES session key is never transmitted in plaintext. It is encrypted with the server's RSA-2048 public key, ensuring that only the legitimate server can recover it.
3. **Data at Rest:** User credentials are stored as salted SHA-256 hashes, protecting them even if the database file is exfiltrated. RSA private keys are stored on disk in PEM format, with access controlled by file-system permissions.
4. **Authentication:** Users must register and log in before accessing any service. The server rejects duplicate sessions, preventing impersonation or session hijacking.
5. **Per-Client Isolation:** Each client has its own AES session key and cipher instance. A compromise of one client's key does not affect other clients' sessions.

3.6 Testing

3.6.1 Types of Tests Conducted

Table 2: Test plan summary

Test Type	Scope	Description
Unit Testing	Individual modules	Encrypt/decrypt round-trip, hash verification, register/login, key generation.
Integration Testing	Module interaction	Full handshake: auth → key exchange → encrypted messaging.
Functional Testing	End-to-end	Multi-client chat: messages sent by one client are correctly received and displayed by others.
Security Testing	Cryptographic correctness	Tampered ciphertext raises exceptions; wrong passwords are rejected; duplicate logins are blocked.
GUI Testing	User interface	Login/Register flow, message send/receive, hash computation, AES encrypt/decrypt tool.

3.6.2 Sample Test Scenarios

1. **AES Round-Trip:** Encrypt a plaintext message, then decrypt the result; verify the recovered plaintext matches the original.
2. **Tamper Detection:** Flip a bit in the ciphertext and attempt decryption; verify that a `ValueError` is raised due to tag mismatch.
3. **RSA Key Exchange:** Generate a key pair, encrypt a 16-byte AES key with the public key, decrypt with the private key, and verify equality.
4. **Password Salting:** Register two users with the same password; verify that the stored hashes differ (because the salts differ).
5. **Duplicate Login Prevention:** Log in as a user, then attempt a second login with the same username from a different client; verify the server returns `DUP`.
6. **Multi-Client Broadcast:** Connect three clients, send a message from one, and verify that the other two receive it correctly decrypted.

3.6.3 Phase 4 Summary

By the end of Phase 4, the Secure Communication Suite is a fully integrated application that:

- Authenticates users via password-based credentials.
- Establishes secure sessions using RSA key exchange.
- Encrypts all communication with AES-EAX authenticated encryption.
- Provides SHA-256 hashing for independent data integrity verification.
- Offers a modern graphical interface for user-friendly interaction.

4 Conclusion

The Secure Communication Suite successfully demonstrates the practical application of fundamental cryptographic techniques in a real-world Python application. Across three development phases (Phases 2–4), the project incrementally built, integrated, and tested six core security modules:

- **AES-EAX** for authenticated symmetric encryption,
- **RSA-OAEP** for asymmetric key exchange,
- **SHA-256** for data integrity verification,
- **Key Management** for secure key lifecycle handling,
- **Password Authentication** with salted hashing,
- **Internet Services Security** via a multi-threaded client–server architecture.

The hybrid encryption approach (RSA for key transport, AES for bulk encryption) mirrors real-world protocols such as TLS, providing students with hands-on experience with industry-standard security practices.

References

[1] NIST, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001.

[2] R. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[3] NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[4] M. Bellare, P. Rogaway, and D. Wagner, "The EAX Mode of Operation," *FSE 2004*, LNCS 3017, pp. 389–407, 2004.

[5] PyCryptodome Documentation, https://www.pycryptodome.org/.

[6] CustomTkinter Documentation, https://customtkinter.tomschimansky.com/.